`microgpt.py` — An Annotated Guide

Training a GPT from Scratch in Pure Python

@karpathy (original)
@nealrichter + Amazon Kiro (annotations)

June 29, 2026

Abstract

This document walks through `microgpt.py`, a minimal implementation of GPT pretraining and inference in pure Python with zero dependencies. Each section presents the code alongside its mathematical foundations and references to the original papers. The goal is to make the complete algorithm—tokenization, autograd, transformer forward pass, Adam optimization, and autoregressive sampling—accessible to anyone with basic Python and linear algebra knowledge.

This is part of a three-stage teaching project: **pretrain** (this file) → **LoRA SFT** (`microgpt_sft.py`) → **DPO alignment** (`microgpt_dpo.py`), mirroring how production LLMs are built. An optional visualization module (`microgpt_viz.py`, invoked via `--viz`) renders live ASCII loss sparklines, attention heat maps, and embedding similarity grids so you can *see* what the model learns. An example training run is included as an appendix.

Source: `https://github.com/nealrichter/research/blob/main/microgpt/microgpt.py`

Contents

1 Overview

`microgpt.py` implements the full GPT training pipeline in ∼276 lines of pure Python:

1. **Data**: Download and load a corpus of 32,000 names, tokenize each character to an integer.
2. **Autograd**: Build a scalar-valued computation graph with automatic differentiation.
3. **Model**: Feed tokens through a GPT-2-style transformer (multi-head attention + MLP).
4. **Training**: Compute cross-entropy loss, backpropagate gradients, update with Adam.
5. **Inference**: Sample new names autoregressively with temperature scaling.

Parameter	Value
Layers	1
Embedding dimension (d)	16
Attention heads (h)	4
Per-head dimension (d_k)	4
Context length (block size)	16
MLP hidden dimension	64
Total parameters	∼4,200
Training steps	1,000

Table 1: Model architecture. Notable differences from GPT-2: RMSNorm instead of LayerNorm, no biases, ReLU instead of GeLU.

2 Data and Tokenization

The model trains on a character-level corpus of human names. Each unique character in the dataset becomes a token ID (0 through 25 for a–z), plus a special Beginning-of-Sequence (BOS) token at index 26. This gives a vocabulary of 27 tokens.

```
# Let there be a Dataset `docs`: list[str] of documents
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)

# Let there be a Tokenizer
uchars = sorted(set(''.join(docs)))
BOS = len(uchars)
vocab_size = len(uchars) + 1
```

The tokenizer is minimal: `encode(c) = uchars.index(c)` maps a character to its integer ID, and `decode(i) = uchars[i]` reverses it. Each document (name) is wrapped with BOS tokens on both ends:

$$[\text{BOS}, t_1, t_2, \ldots, t_n, \text{BOS}]$$

The trailing BOS acts as an end-of-sequence signal. During training, the model learns to predict each next token given the preceding context.

3 Autograd Engine

The heart of the implementation is the `Value` class: a scalar-valued node in a computation graph that supports reverse-mode automatic differentiation (backpropagation) [4]. Every arithmetic operation builds the graph forward; calling `.backward()` traverses it in reverse to compute gradients.

```python
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads')

    def __init__(self, data, children=(), local_grads=()):
        self.data = data # forward value
        self.grad = 0 # dL/d(this node)
        self._children = children # parent nodes
        self._local_grads = local_grads # d(this)/d(child_i)
```

Each node stores four things: its scalar `data` (computed during the forward pass), its `grad` (filled during backward), the nodes it was computed *from* (`_children`), and the local partial derivatives relating it to those children.

3.1 Forward Operations

Every operation creates a new `Value` and records the local derivatives needed for the chain rule:

```python
    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other):
        return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self):
        return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self):
        return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self):
        return Value(max(0, self.data), (self,), (float(self.data > 0),))
```

The local gradients follow directly from single-variable calculus:

$$\frac{\partial(a+b)}{\partial a} = 1 \qquad \frac{\partial(ab)}{\partial a} = b \qquad \frac{\partial(x^n)}{\partial x} = nx^{n-1} \tag{1}$$

$$\frac{\partial \ln x}{\partial x} = \frac{1}{x} \qquad \frac{\partial e^x}{\partial x} = e^x \qquad \frac{\partial \text{ReLU}(x)}{\partial x} = \mathbf{1}[x > 0] \tag{2}$$

3.2 Backward Pass (Backpropagation)

The backward pass computes $\partial \mathcal{L}/\partial v$ for every node v in the graph. It first builds a topological ordering (children before parents), then walks it in reverse, applying the chain rule at each node:

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._children:
                build_topo(child)
            topo.append(v)
    build_topo(self)
    self.grad = 1
    for v in reversed(topo):
        for child, local_grad in zip(v._children, v._local_grads):
            child.grad += local_grad * v.grad
```

The key line is the last one. For node v with child c_i:

$$\frac{\partial \mathcal{L}}{\partial c_i} \mathrel{+}= \frac{\partial v}{\partial c_i} \cdot \frac{\partial \mathcal{L}}{\partial v}$$

The "+ =" accumulates contributions from multiple paths through the graph (a node may be used more than once).

4 Model Architecture

The architecture follows GPT-2 [2] with three simplifications: RMSNorm [5] replaces LayerNorm (simpler, no mean subtraction), all bias terms are removed, and GeLU is replaced with ReLU.

4.1 Parameter Initialization

All weight matrices are initialized from $\mathcal{N}(0, 0.08^2)$. The model has three embedding tables (`wte` for tokens, `wpe` for positions, `lm_head` for the output projection) plus six matrices per transformer layer (Q, K, V, O projections for attention, and two MLP layers):

```
n_layer = 1
n_embd = 16
block_size = 16
n_head = 4
head_dim = n_embd // n_head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std))
    for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd),
              'wpe': matrix(block_size, n_embd),
              'lm_head': matrix(vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
```

4.2 Building Blocks

4.2.1 Linear Layer

A simple matrix-vector multiply $y = Wx$ (no bias):

```
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]
```

4.2.2 Softmax

The numerically stable softmax subtracts the maximum before exponentiating to prevent overflow:

```
def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]
```

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}}$$

4.2.3 RMSNorm

Root Mean Square normalization [5] scales the input by the inverse RMS, without subtracting the mean (unlike LayerNorm):

```
def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

$$\text{RMSNorm}(x_i) = \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}}$$

4.3 Transformer Forward Pass

The `gpt()` function processes one token at a time, maintaining a KV-cache so that at position t only the new key/value vectors are computed (past ones are reused). This makes autoregressive generation efficient.

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id]
    pos_emb = state_dict['wpe'][pos_id]
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x)
```

The input to the transformer is the sum of a token embedding and a learned positional embedding, followed by RMSNorm:

$$x^{(0)} = \text{RMSNorm}(W_{\text{te}}[\text{token}] + W_{\text{pe}}[\text{pos}])$$

4.3.1 Multi-Head Attention

The attention mechanism [1] allows each position to attend to all previous positions. The embedding is split into $h = 4$ heads of dimension $d_k = 4$ each. For each head:

```
    for li in range(n_layer):
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j]
                for j in range(head_dim)) / head_dim**0.5
                for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j]
                for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
```

The scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The $\sqrt{d_k}$ scaling prevents the dot products from growing large in magnitude, which would push softmax into regions with vanishing gradients. Causal masking is implicit: the KV-cache only contains past positions, so future tokens are never attended to.

After attention, an output projection W_O recombines the heads, and a residual connection [6] adds the input back.

4.3.2 MLP Block

A two-layer feed-forward network with ReLU activation and 4× expansion:

```
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]
```

$$\text{MLP}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot \text{RMSNorm}(x)) + x$$

GPT-2 uses GeLU [7]; ReLU is substituted here for simplicity (one fewer special function in the autograd).

4.3.3 Output Head

The final linear layer projects from embedding space ($d = 16$) to vocabulary logits ($V = 27$):

```
    logits = linear(x, state_dict['lm_head'])
    return logits
```

5 Training

5.1 Loss Function

The training objective is the standard language modeling loss: cross-entropy over next-token prediction, averaged across the sequence:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} \log P_\theta(t_{i+1} \mid t_1, \ldots, t_i)$$

Each training step processes one name. The model predicts the probability distribution over the vocabulary at each position, and we take the negative log probability of the correct next token:

```
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses)
```

After computing the loss, `loss.backward()` traverses the entire computation graph (thousands of `Value` nodes per step) to fill in gradients for every parameter.

5.2 Adam Optimizer

The model is optimized with Adam [3]—the de-facto standard for training neural networks. Adam maintains exponential moving averages of the gradient (first moment m) and the squared gradient (second moment v), with bias correction to account for their initialization at zero:

```
    lr_t = learning_rate * (1 - step / num_steps)
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0
```

The full Adam update:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{3}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{4}$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t) \tag{5}$$

$$\theta_t = \theta_{t-1} - \eta_t \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon) \tag{6}$$

The learning rate decays linearly from 0.01 to 0 over 1,000 steps: $\eta_t = 0.01 \cdot (1 - t/1000)$.

6 Inference

After training, the model generates new names by sampling one token at a time. At each position, it feeds the current token through the transformer, applies temperature-scaled softmax to get a probability distribution, and samples the next token:

```
temperature = 0.5
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size),
                                  weights=[p.data for p in probs])[0]
        if token_id >= len(uchars):
            break
        sample.append(uchars[token_id])
```

Temperature $\tau \in (0, 1]$ controls the sharpness of the sampling distribution:

$$P(t_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

- $\tau \to 0$: greedy (always picks the highest-probability token).
- $\tau = 1$: original distribution (maximum diversity).
- $\tau = 0.5$ (used here): a balance between coherence and variety.

Generation stops when the model emits a special token (BOS or any ID $\geq$ the character vocabulary size).

7 Model Serialization

After training, the full model state (vocabulary, architecture config, and all weight values) is saved as a JSON file:

```
    with open('model.json', 'w') as f:
        json.dump({'vocab': uchars,
                'config': {'n_layer': n_layer, 'n_embd': n_embd,
                        'block_size': block_size, 'n_head': n_head},
                'weights': {k: [[p.data for p in row] for row in mat]
                        for k, mat in state_dict.items()}}, f)
```

This enables inference-only mode (`-i`), which loads the saved model and generates samples without retraining. It also serves as the input for the next pipeline stage (`microgpt_sft.py`).

8 Usage

```
python3 microgpt.py # pretrain -> model.json
python3 microgpt.py -i # inference from model.json
python3 microgpt.py -i model_sft.json # inference from any saved model
python3 microgpt.py --viz 250 # train with visualization every 250 steps
python3 microgpt.py -h # help
```

References

[1] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[2] Radford, A., Wu, J., Child, R., et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[3] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. ICLR, 2015. https://arxiv.org/abs/1412.6980

[4] Rumelhart, D.E., Hinton, G.E., and Williams, R.J. *Learning Representations by Back-propagating Errors*. Nature 323, 1986.

[5] Zhang, B. and Sennrich, R. *Root Mean Square Layer Normalization*. NeurIPS, 2019. https://arxiv.org/abs/1910.07467

[6] He, K., Zhang, X., Ren, S., and Sun, J. *Deep Residual Learning for Image Recognition*. CVPR, 2016. https://arxiv.org/abs/1512.03385

[7] Hendrycks, D. and Gimpel, K. *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415, 2016. https://arxiv.org/abs/1606.08415

Appendix: Example Training Run

TBA